

Development of a Plug-In Electric Vehicle (PEV) Simulator for Charging Station (EVSE) Testing

Gregor Sellschopp

This document is a machine translated version of this file:

https://github.com/gregsell/DieterEvDriver/blob/main/docs/thesis_CCS_PEV_SIM_german.pdf

Some of the links broke during the translation, and a lot of weird formatting bugs were introduced. For sure I missed some of them. Please do not copy any configuration from here but refer to the repository instead.

Abstract

The Combined Charging System (CCS) is the dominant standard for DC fast charging of electric vehicles in Europe, combining analog Control Pilot signaling with Power Line Communication for high-level protocol exchange.

Testing charging infrastructure typically requires a physical vehicle. Existing open-source implementations focus on the charging station side and do not address vehicle-side emulation.

This thesis develops an integrated PEV simulator based on the EVerest framework, covering the full CCS charging sequence from IEC 61851 and SLAC through ISO 15118-2. A custom interface PCB handles low-level control via a microcontroller, while a custom EVerest module bridges the hardware and protocol stack. A deployable Yocto image for the Raspberry Pi is produced to ensure a portable and reproducible setup.

Both AC and DC charging sessions are successfully demonstrated. EVerest proves to be flexible and capable, though complex to implement. Open issues include a heartbeat mechanism for safe shutdown and support for higher-current contactors.

Table of Contents

List of Abbreviations	VII
List of Figures	IX
List of Tables.....	XI
1 Introduction.....	1
2 State of the Art.....	3
2.1 DC Fast Charging with CCS	3
2.1.1 Architecture and Connector Standard.....	3
2.1.2 Control Pilot and Charging Process.....	5
2.1.3 Classification in the OSI Model	6
2.1.4 Comparison with Other Standards.....	7
2.2 Existing CCS Implementations	8
2.2.1 pyPLC / foccci.....	8
2.2.2 EVerest.....	8
3 Method.....	13
3.1 Requirements.....	13
3.1.1 Functional	13
3.1.2 Non-functional.....	13
3.2 System Architecture.....	14
3.2.1 Hardware	14
3.2.2 Software.....	15
3.3 Component Selection and Design	16
3.3.1 CCS Connectors.....	16
3.3.2 CP Measurement and Control	17
3.3.3 Power Measurement.....	18
3.3.4 Power Isolators and Relays	21
3.3.5 SBC and Processor	22
3.3.6 PCB	22
3.4 Dieter Firmware	24
3.4.1 Commands	25
3.4.2 Measurement Values.....	26
3.5 EVerest	27
3.5.1 The ev_board_support Interface.....	28
3.5.2 DieterEvDriver Module.....	29

3.5.3	Runtime Configuration	31
3.6	Build and Development Tools	31
3.6.1	CMake	31
3.6.2	Yocto.....	32
3.6.3	Open-plc-utils.....	34
3.6.4	Venv & ev-cli	35
3.6.5	Admin Panel	35
3.6.6	MQTT Explorer	36
3.6.7	Wireshark.....	36
3.6.8	PlatformIO.....	37
4	Results.....	39
5	Summary, Conclusions, and Outlook	43
6	Bibliography.....	44
	Appendix	49

List of Abbreviations

AC	Alternating Current
ADC	Analog-to-Digital Converter
BSP	Board Support Package
CCS	Combined Charging System
CLI	Command Line Interface
CMD	Command
CP	Control Pilot
DC	Direct Current
DHCP	Dynamic Host Configuration Protocol
DIETER	Device Interface ElecTronic (Especially not limited to Raspberry)
EV	Electric Vehicle
EVSE	Electric Vehicle Supply Equipment
GB/T	National Standard Recommended (Chinese equivalent)
HLC	High-Level Communication
HV	High Voltage
LDO	Low Dropout Regulator
LFE	Linux Foundation Energy
MCU	Microcontroller Unit
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MQTT	Message Queuing Telemetry Transport
NACS	North American Charging Standard
OCPP	Open Charge Point Protocol
OSI	Open System Interconnection model
PE	Protective Earth
PEV	Plug-in Electric Vehicle
PLC	Power Line Communication
PP	Proximity Pilot
PV	Photovoltaics
PWM	Pulse-width modulation
SBC	Single-Board Computer
SLAC	Signal Level Attenuation Characterization
UART	Universal Asynchronous Receiver-Transmitter
VAR	Variable
V2G	Vehicle-to-Grid

List of Figures

Figure 2.1:	Type 2 Combo Connector (vehicle side) [1]	3
Figure 2.2:	Type 1 Combo Connector [2]	4
Figure 2.3:	Resistor network for AC charging [3, p. 28]	5
Figure 2.4:	V2G OSI [3, p. 43]	7
Figure 3.1:	Hardware architecture, based on [22]	14
Figure 3.2:	Software architecture.....	15
Figure 3.3:	Schematic diagram for CP measurement and control.....	18
Figure 3.4:	Circuit Diagram for Processing an Analog Signal.....	20
Figure 3.5:	Circuit diagram for voltage measurement.....	21
Figure 3.6:	Visualization of the PCB	22
Figure 3.7:	Incorrect IP assignment of the Homeplug modem	37
Figure 4.1	Construction of the vehicle simulator.....	40
Figure 4.2:	Making tea on the Alpitronic HYC-300	41
Figure B.1:	Dieter's Circuit Diagram.....	52
Figure C.1	PCB Dimensions	53

List of Tables

Table 2.1:	Combo-2 Contact Designations [1].....	4
Table 2.2:	Comparison of CCS Specifications [1], [2]	4
Table 2.3:	CP States [3, p. 28]	6
Table 3.1:	Bidirectional Motor Control, according to [26, p. 9].....	17
Table 3.2:	PCB Function Blocks.....	23
Table 3.3:	Key Terms Related to Insulation [27]	23
Table 3.4:	MCU–Linux Protocol	24
Table 3.5:	Relevant data sheet specifications for the locking actuator [23, p. 3].....	25
Table 3.6:	Commands not supported by DieterEvDriver	29
Table 3.7:	Commands supported by DieterEvDriver	29

1 Introduction

In recent years, electric mobility has evolved from a niche technology into a central component of the transportation transition. In Europe, the Combined Charging System (CCS) has established itself as the dominant standard for DC fast charging. It enables charging powers of several hundred kilowatts and relies on the interaction of multiple protocols.

Interoperability between the vehicle and the charging station is by no means a given. Although the standard exists, actual implementations are sometimes proprietary, lack transparency, and often do not comply with the standard.

For the development and commissioning of charging infrastructure, it is necessary to emulate the vehicle-side portion of the protocol. This enables easy testing of the charging station for basic functionality and special situations. Existing open-source implementations focus on the charging station side.

The goal of this thesis is to develop such a PEV simulator. The emulator is intended to fully replicate the CCS charging process—from IEC 61851 signaling through the SLAC protocol to ISO 15118-2 communication—while being integrated into a standalone hardware unit.

The following sections first present the relevant technical fundamentals of CCS as well as existing implementations. Chapter 3 describes the methodological approach, ranging from requirements through hardware and software architecture to the implementation of the individual components. The results obtained are presented in Chapter 4, summarized in Chapter 5, and followed by an outlook on remaining tasks.

2 State of the Art

2.1 DC Fast Charging with CCS

The Combined Charging System (CCS) is an international standard for charging electric vehicles. It combines various interfaces that previously had different applications: single-phase and multi-phase AC charging and DC fast charging. Implementing these requirements results in a rather complex charging process, which will be briefly explained in this chapter.

2.1.1 Architecture and Connector Standard

An important requirement in the design of the connector was backward compatibility with the respective predecessor standards. In Europe, this is IEC 62196 Type 2 (also known as IEC Type 2), which was established in 2013 and governs AC charging with up to three phases. Due to its widespread use, this connector was adopted for CCS (specified in IEC 62196-3) and expanded to include two additional, stronger contacts for DC charging. This combination is referred to as the “Combo-2” connector standard. Often, the unwanted contacts for AC charging are omitted.

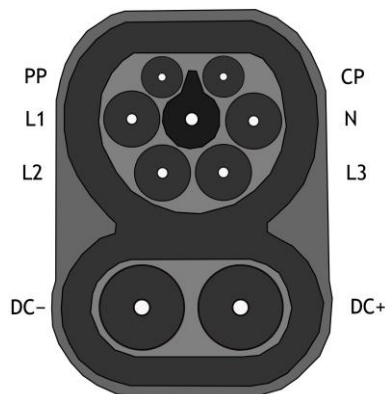


Figure 2.1 : Type 2 Combo Plug (vehicle side) [1]

Table 2 .1 : Combo-2 pontact pinout [1]

Abbreviation	Description
PE	Protective conductor
CP	Communication between the vehicle and the charging station via an analog signal
PP	Limiting the charging current via resistance coding
DC+	DC charging, positive terminal
DC-	DC charging, negative terminal

It should be noted that the terms are often used incorrectly: “CCS” itself does not refer to the connector, but specifically to the communication protocol. For example, the CCS protocol was also adopted in North America, but with a different connector. There, the Type 1 vehicle connector (SAE J1772) was used as the basis, which is why the CCS design is known as “Combo 1.”



Figure 2 .2 : Type 1 Combo Connector [2]

Historically, two CCS specifications were developed; their key data are shown in Table 2 .2 .

Table 2 .2 : Comparison of CCS Specifications [1], [2]

	CCS 1.0	CCS 2.0
Year of Release	2013	2016
Voltage	max. 500 V	200–1,000 V
Current	Max. 400 A	Max. 500 A
Power	max. 200 kW	Max. 500 kW
Connection	Combo 1 or 2 (IEC 62196-3)	
Communication	DIN SPEC 70121	DIN SPEC 70121 ISO 15118
Load distribution	Reactive	Reactive and planned
Activation	External payment	External payment / Plug&Charge

This also shows that the common designation of the Combo-2 connector combination as a “CCS2 connector” is incorrect, because it does not refer to the specification of the physical connector.

Communication can be divided hierarchically into two parts: So-called low-level communication, which contains the logic for AC charging, was adopted from CCS and runs through the CP and PP contacts. The vehicle can then signal to the EVSE (Electric Vehicle Supply Equipment—charging station) to switch to DC charging, thereby initiating the negotiation of parameters for CCS.

2.1.2 Control Pilot and Charging Process

During AC charging, the relevant connections can be illustrated as follows:

Since the voltage is known during AC charging, the vehicle only needs to be informed of the maximum power it is allowed to draw. For this purpose, a 1 kHz PWM signal (+/- 12 V) is applied between the CP and PE contacts. The charging station adjusts the duty cycle depending on the permissible current. For example, while a 60% duty cycle corresponds to a permissible current of 36 A, the vehicle must limit itself to 18 A at a 30% duty cycle. [3, p. 27]

If, on the other hand, charging is to be performed with direct current, a duty cycle of 5% is generated, thereby initiating the switch to power line communication (PLC). This is referred to below as high-level communication (HLC) and is explained in the section 2.1.3.

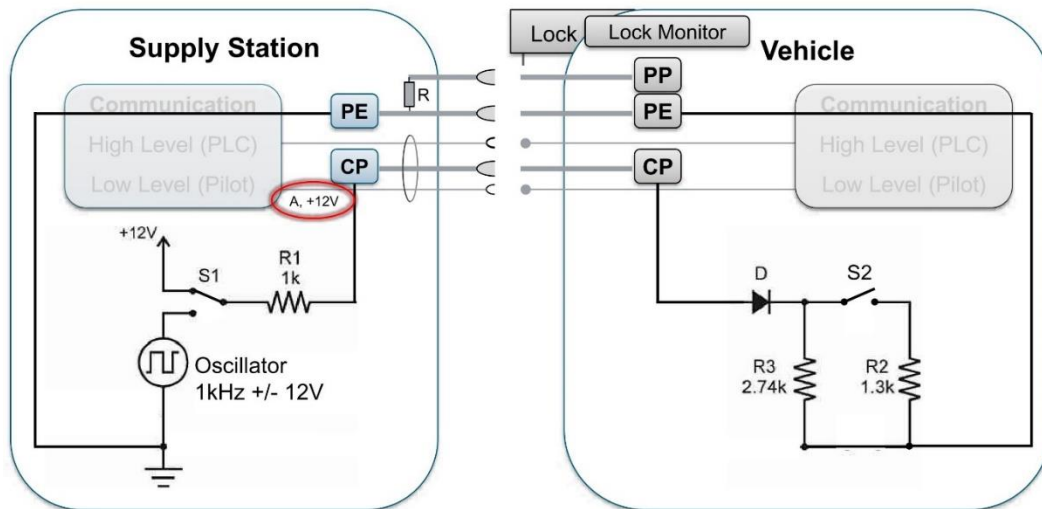


Figure 2.3 : Resistor Network for AC Charging [3, p. 28]

The charging station can determine the vehicle's state based on the voltage level at the CP contact. As soon as a vehicle is connected, the voltage drops to 9 V due to the resistance network that is formed. If the vehicle additionally closes S2, it signals readiness. IEC 61851-1 standardizes six states that represent the vehicle's status using the CP line:

Table 2 .3 : CP States [3, p. 28]

State	Voltage	Description
A	+12 V	- No vehicle connected
		- Connection to vehicle established
B	+9 V	- Vehicle not ready
		- Charging station power off
C	+6 V	- Vehicle ready
		- Charging station power on
D	+3 V	- Vehicle ready
		- Charging station power on
		- Ventilation required
E	0 V	- Short circuit between CP and PE
		- Unlock after max. 30 ms
F	-12 V	- Charging station not ready

2.1.3 Classification in the OSI Model

All communication is based on the seven-layer OSI model, with each layer performing a specific process for the hierarchically higher layer. This works by each layer prepending its header to the message.

The connection between CP and PE and the signaling of 5% PWM on CP constitute Layer 1. This fulfills the prerequisite for Layer 2: the configuration of the PLC modems and the startup of the HLC. A process characteristic of this procedure is the so-called Signal Level Attenuation Characterization (SLAC). Here, the signal strength is used to ensure that a vehicle connects via PLC to the charging station it is actually connected to and not to the neighboring device, since electromagnetic interference can cause crosstalk.

Several additional layers are required between the physical layer and the application layer, but these are not relevant to this work. Among others, these include TCP and TLS for encrypted communication, EXI for compact data transmission, and the V2G session layer. [3, pp. 40–54]

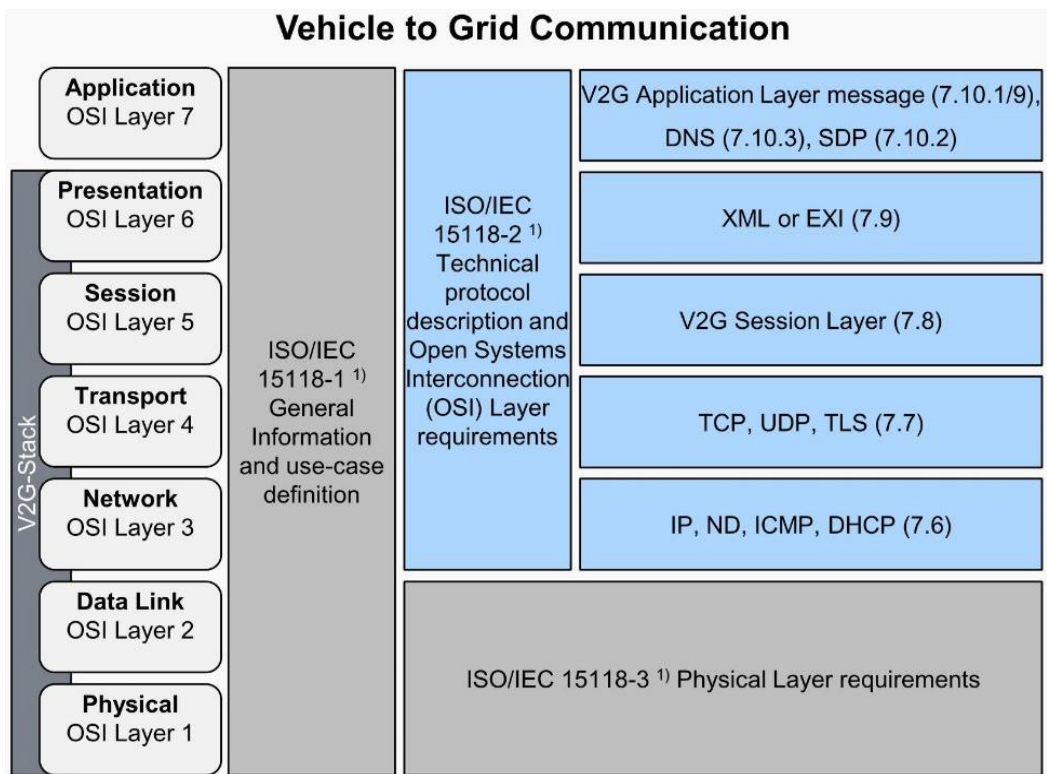


Figure 2 .4 : V2G OSI [3, p. 43]

2.1.4 Comparison with Other Standards

Of course, there are other ways to charge vehicles with direct current. Before the introduction of CCS, the CHAdeMO standard was the most prominent in Europe. It was primarily promoted by Japanese manufacturers, but it had one major drawback: AC charging is not possible, which is why a separate plug is required for that purpose. Other distinguishing features include additional contacts for controlling the vehicle’s contactors and a dedicated power supply for auxiliary consumers.[4] Although communication is based on CAN—unlike CCS—and can therefore be implemented much more easily¹, CHAdeMO has not gained widespread acceptance: as of 2026, out of nearly 200,000 charging points in Germany, only about 3,300 were equipped with a CHAdeMO connection. [6]

Nevertheless, the basic idea behind the CAN implementation has endured and is part of the GB/T standard, which dominates in China. Compared to CCS, this standard has a significantly higher minimum voltage of 750 V and also features a separate pair of cables for data transmission as well as CP/PP counterparts.[1] ,[7]

The third internationally relevant solution is the North American Charging Standard (NACS) developed by Tesla, which has also adopted the CCS protocol but with its own, more compact connector. [8]

¹ According to[5] , implementation effort is up to twenty times lower

2.2 Existing CCS Implementations

2.2.1 pyPLC / foccci

The first open-source project for CCS implementation originated from the openinverter forum in early 2022. The project aimed to capture the communication between the charging station and the vehicle using commercially available HomePlug PLC adapters. This required adding several electronic components, reprogramming the modems using an open-source toolkit from Qualcomm Atheros, and an external power supply.[9], [10] , [11]

GitHub user uhi22 built upon this foundation and wrote the Python software pyPLC, which runs on both Linux and Microsoft Windows.[11] The goal was to monitor all communication, as well as to simulate both the vehicle and the charging station sides.[12] The basic idea was to create two separate hardware modules: one for low-level communication via the CP pin and one for high-level communication via Ethernet using a PLC. While the HomePlug modem was connected to the computer via Ethernet, a separate electronic circuit was developed for, among other things, switching the CP MOSFET (S2 in Figure 2.3 : Resistor Network for AC Charging [3, p. 28]) and controlling the contactors. This circuit was based on an Arduino and communicated with the main computer (often a Raspberry Pi) via UART. [11], [13]

The project was so successful that, as a result, a dedicated—and likewise open-source—control unit was developed in collaboration with the community on openinverter.org to retrofit vehicles with CCS. The Fully Open-source CCS Controller Interface (FOCCCI) is a fully integrated board that includes the CP circuitry and a PLC modem, as well as, for example, drivers for power contactors and a CAN interface to the vehicle² .[14] [15] The software is a version of pyPLC ported for microcontrollers.[12] The entire development was presented at the Augsburg Linux Info Tag in 2024. [5]

Despite the generally good documentation, this project did not seem to be the best option for building a PEV simulator. The main reason was that the implementation of the CCS protocol is quite complex, and there is significant uncertainty regarding compliance with standards. Furthermore, future support is questionable given the small scope of the project. Nevertheless, it provided a great deal of useful information and was regularly used as a reference in this thesis.

2.2.2 EVerest

Unlike pyPLC, the EVerest project is significantly larger in scope. Since its launch by Pionix GmbH, many individuals, universities, and other companies have contributed code. Although the main development takes place in Germany, it is used worldwide for EVSE development, in part due to its integration as part of the Linux Foundation Energy (LFE). EVerest runs on any device and can be used both for developing standalone AC chargers for home use and for complex, branched public DC charging stations.

The software supports all standards to enable any communication between EVs and EVSEs, as well as with PV components and OCPP. The motivation behind the project was to optimize EVSE

² Sold, among other places, in the [openinverter shop](#)

development. The current state of the art is that each EVSE manufacturer develops its own hardware and software to incorporate custom functionalities. Depending on the application, many interfaces must be taken into account, including:

- Charging controllers
- OCPP
- Electricity meters
- DC power supplies
- UI elements: RFID/NFC readers, screens
- Payment terminals

The many possible combinations require a significant amount of development effort, even though the functionality is often the same.

Architecture

The goal of EVerest is to abstract the logic from the hardware to avoid duplicate development. The implementation of the various protocols is structured into software modules that communicate via defined interfaces (hereinafter referred to as “interfaces”). If a company now wants to build an EVSE using hardware components A, B, and C, it only needs to implement the respective interfaces for interaction with EVerest, but can simply adopt the main logic.

MQTT is used as the information bus, with each module instance running as its own process. Communication between the modules occurs by publishing the variables defined in the module interface with the corresponding topic. Similarly, Module A can trigger a command in Module B via an MQTT command. These two data types are referred to in EVerest as CMD (“command”) and VAR (“variable”). [16]

Interfaces

In EVerest, defining a set of CMDs and VARs forms an interface. Additionally, standardized errors can be defined. For a simple hypothetical power supply, the interface might look like this, for example: Turning the power on and off would be implemented as a CMD, callable by other modules. The device regularly publishes the measured current and voltage values as VARs via MQTT. Other modules that require this data could subscribe to these VARs and monitor them over time.

Each interface is defined via a YAML configuration file. It has the following format: [17]

```
description: >-
2.   Brief description of what this interface does
3.
4. cmds:
5.   command_name:
6.     description: What this command does
7.     arguments:
8.       arg_name:
9.         description: Argument description
10.        type: string
11.   result:
12.     description: Return value description
13.     type: boolean
14.
15. vars:
16.   variable_name:
17.     description: What this variable represents
18.     type: object
19.     $ref: /some_type#/TypeName
20.
21. errors:
22.   - reference: /errors/error_category
```

Modules

In EVerest, modules are defined via a set of implemented interfaces. These interfaces can either be provided to other modules or required by other modules. Each implementation has a unique ID, since an interface can occur multiple times.

Consider a router as an example. It does not simply implement the IP protocol but has multiple instances of it (multiple Ethernet ports, Wi-Fi antennas, etc.), which can be used for various purposes.

Each created module also has a unique ID, since modules can be instantiated as many times as desired. If, for example, a command is to be invoked via MQTT, both IDs must therefore be taken into account. The MQTT topics are respectively

```
1. Everest/{Module Instance ID}/{Interface Implementation ID}/cmd
2. Everest/{Module Instance ID}/{Interface Implementation ID}/var
```

Just like interfaces, modules are defined by a file (manifest.yaml). This file primarily specifies two things: which interfaces a module provides and requires, as well as possible configuration values (these are set in the runtime configuration; see the section “Runtime Configuration”)

The manifest.yaml file has the following format: [18], [19]

```
1. description: Module description
2. config:
3.   parameter_name:
4.     description: Parameter description
5.     type: string|integer|boolean|number
6.     default: value
7. provides:
8.   interface_name:
9.     interface: interface_type
10.    description: What this interface provides
11. requires:
12.   dependency_name:
13.     interface: interface_type
14.     min_connections: 0
15.     max_connections: 1
16. metadata:
17.   license: License URL
18.   authors:
19.     - Author name
```

Everest provides many ready-to-use modules. These contain frequently needed functionality, such as communication with an electricity meter. In this work, modules that interact with hardware (i.e., those that implement the `board_support` interface) will be of greater importance. Such modules already exist for commercial hardware, such as PhyVersoBSP for the Phytex charge controller.[20] In this thesis, an Everest module will be developed that also implements the `ev_board_support` interface and

Runtime Configuration

Runtime configuration is another YAML file that defines the structure of the Everest instance being built. The following items are specified for the modules to be generated:

- Their object IDs
- Which module they implement
- Configuration values
- For each interface requirement of the instantiated module:
 - o The object ID of the module instance that provides the interface
 - o ID of the interface implementation within the module providing the interface

An example configuration can be found in Appendix A .

Relevant modules for the EV side

In principle, EVerest is designed primarily for the EVSE side. The `EvseManager` is the central module here and manages tokens, power meters, and modules for hardware support. Its vehicle-side counterpart is the `EvManager`, which also provides an interface for hardware support, as well as the vehicle-side SLAC protocol and ISO 15118-compliant communication with a vehicle. The following modules are available for integration:

- `EvSlac`: Module from EVerest for in-vehicle SLAC communication
- `pyEvJosev`: Python implementation of an ISO 15118-compliant vehicle using the Josev project
- `Store`: A module from EVerest for storing data in `{key}:{value}` format

Additionally, depending on the hardware, a module is required that provides the “`ev_board_support`” interface. Its development is a key part of this work. [21]

3 Method

3.1 Requirements

The following direct requirements regarding functionality are placed on the PEV simulator. These have implications for both hardware and software. Additional requirements arise from the application and criteria such as safety, fault tolerance, and usability.

Last but not least, the total cost of the system is also a factor. Balancing these criteria against availability, lead time, and the quality of the datasheets is not always easy.

3.1.1 Functional

Obviously, the setup must implement a functional CCS interface to communicate with an EVSE and draw electrical power. Since (at least in the first step) no battery will be connected, an artificial load provided externally is required. This requires suitable hardware to connect or disconnect this load from the high-voltage supply. In principle, the goal is not to test the charging station's full performance capabilities, but "only" to test communication. Therefore, in the first step, the setup should be designed for low power ratings (approx. 10 A, 400 V), but with the option to expand to higher voltages (CCS specifies up to 1000 VDC).

For the HLC part of CCS, a PLC interface is required, while low-level communication takes place via manipulation of the CP line. The simulator also requires suitable hardware for this purpose.

In addition, the power consumed by the artificial load should be measured to verify the EVSE's specifications and to prevent the hardware from being overloaded. The measured values should be accessible via a suitable interface for easy further processing.

To realistically simulate the behavior of an EV and ensure the highest possible level of safety, it should be possible to lock the charging cable connector "on the vehicle side." This is typically implemented using a gear motor that pushes a pin straight into a designated recess in the connector, thereby locking it in place.

3.1.2 Non-functional

In addition, there are several other requirements that do not directly affect functionality. As this is a research project, the system should be based on open-source software that, whenever possible, has a broad user base and good future support. The EVerest platform is well-suited for this purpose.

To avoid excessive dependence on specific hardware, the software should operate as independently as possible from the hardware. EVerest excels in this regard as well.

Electrical safety must not be neglected either. Especially given the high voltages permitted in CCS, sufficient creepage distances, galvanic isolation where needed, and, of course, protection against accidental contact must be incorporated into the design.

Furthermore, the option to expand the system to include AC functionality should remain open. As shown in section 2.1.1, this requires additional cables and contactors.

3.2 System Architecture

3.2.1 Hardware

DC charging requires electronics that are connected directly to the CCS data lines and communicate with the EVERest host via a suitable interface. The goal of this implementation is not a fully integrated system, but rather a functional research setup, which is why some off-the-shelf components are used. These include the Homeplug modem (TP-Link TL-PA4010P v5.0), which contains a PLC controller along with accessories. The relevant interfaces are an Ethernet port, which is connected to the SBC, and the CP contact, onto which the HLC is modulated.

In addition, an external motor controller is used to actuate the locking mechanism in the CCS connector. The contactors used to switch the high voltage are also external components.

All other components (circuit for controlling and reading the CP signal, power meter) are integrated into an assembly that will be referred to below as “Dieter” (Device Interface ElecTRonic Especially not limited to Raspberry). Credit: Uwe Hennig (github.com/uhi22) coined this name for the PyPLC project.[11], [13] Since his documentation was very useful and served as the basis for this project, the name was adopted.

The components related to high voltage are installed in the order specified in the CCS Design Guide [3, p. 11] .

An overview of the hardware is shown in Figure 3.1; the isolation boundary between the microcontroller and the high-voltage system is marked in green.

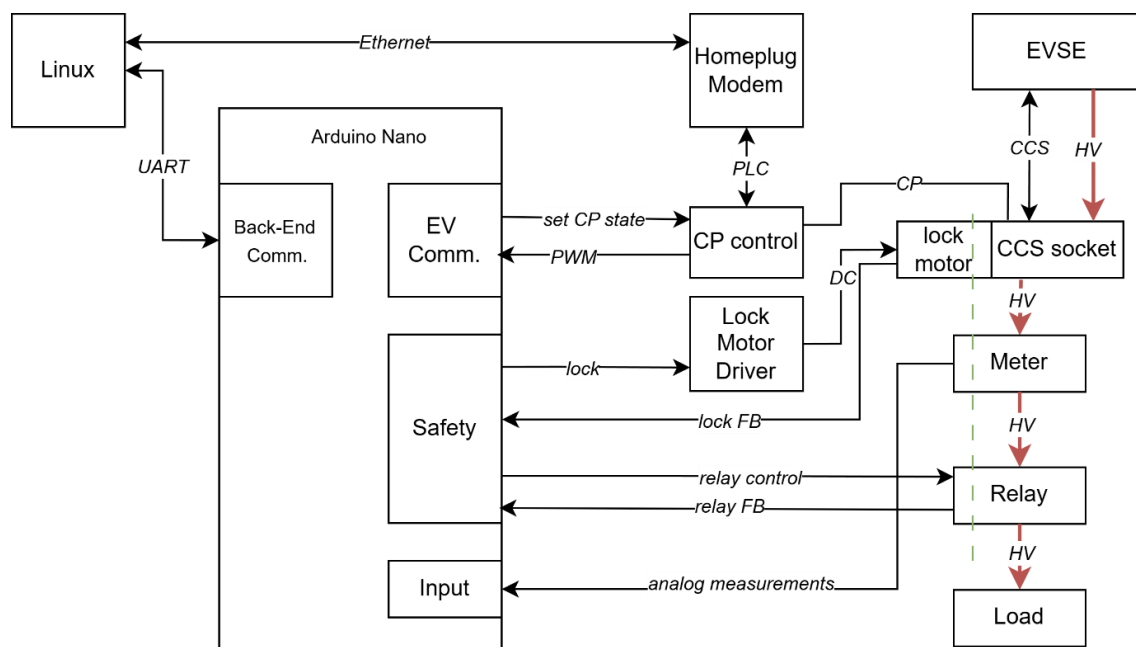


Figure 3 .1 : Hardware architecture, based on [22]

3.2.2 Software

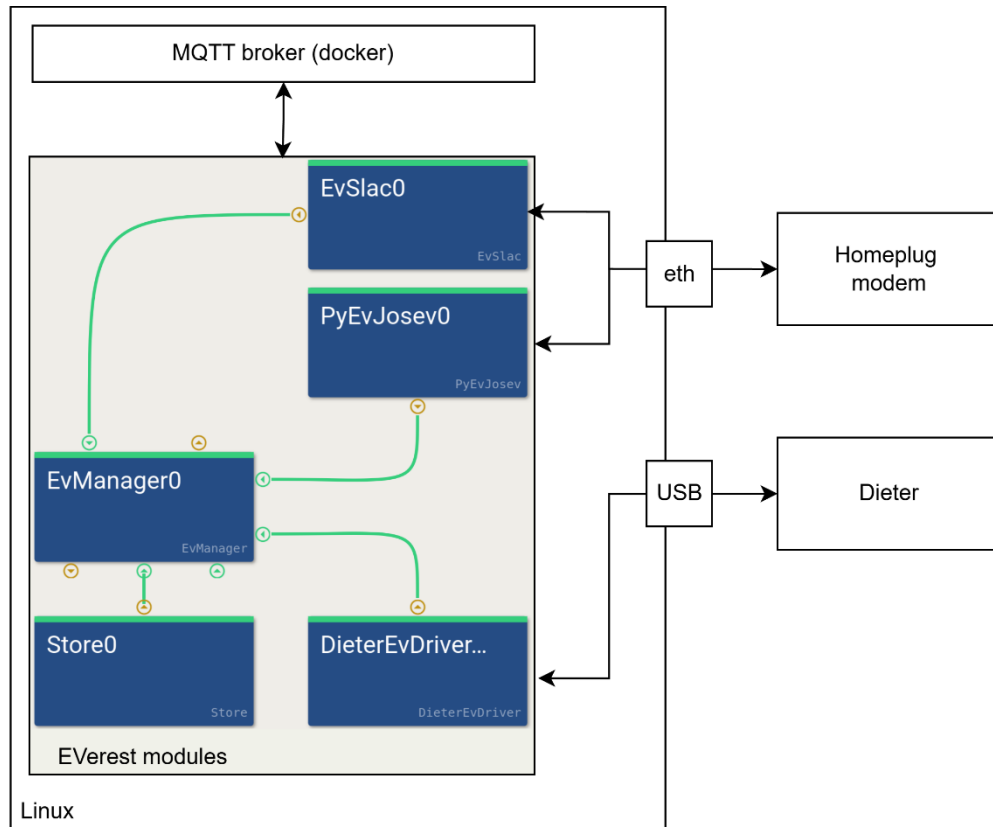


Figure 3 .2 : Software Architecture

The system's software is divided into two parts: on the one hand, the EVERest stack on Linux; on the other, low-level communication via the microcontroller. Data is exchanged asynchronously between the subsystems via UART.

The microcontroller serves purely as an executor; all logic is to be encapsulated in EVERest modules. There are two main reasons for this: First, the system should be more straightforward by implementing logic centrally in one place. This is intended to simplify development and ensure traceability. Second, there are practical considerations. If a change is made, it will not be necessary to modify both EVERest and the “Dieter” firmware.

EVERest

The EVERest module configuration is based on `EvManager` as the central connector. Vehicle simulation is handled by `PyEvJosev` from the EVERest project. Three of the modules have access to hardware: the custom-written `DieterEvDriver` accesses a serial port via `ev_board_support`, while `PyEvJosev` and `EvSlac` use the same Ethernet interface.

`Store` is a helper module that stores key-value pairs for `EvManager`.

3.3 Component Selection and Design

3.3.1 CCS connector

The most obvious component from the outside is the CCS connector. These are very large in order to withstand the required currents. That is also why the high-voltage cables are crimped directly into the connector. It is therefore not possible to reuse a used connector with cables that have been cut too short. [23]

When purchasing a new one, there are two options: the original part from Phoenix Contact for a high three-digit amount[24] or a clone from China for about a quarter of the price. An additional challenge here is that the locking mechanism is not included. Because the mechanical compatibility between the CCS socket and the locking actuator is not immediately apparent and little information on the subject could be found, this option will not be pursued further.

Instead, we'll take advantage of the abundant supply of used CCS plugs. For less than 200 €, you can find some components with a locking mechanism and a few meters of cable (e.g., on ovoko.de). The advantage here is that these are tested original parts, and you save on a crimp connection as well as additional high-voltage cable. In this project, a component from a BMW i3 is used. Due to its age, it has a rated current of "only" 150 A.

Locking Mechanism

During the charging process, the cable must be secured to prevent it from being unplugged under load. The resulting arcing would cause damage to the material and also pose a safety risk.

There are two approaches to locking charging cables into the socket: using a magnetic actuator or a DC motor with a gearbox. In both cases, a pin is inserted into the plug.

As a workaround, there is an option for manual unlocking.

The magnetic actuator offers the advantage of a simpler mechanical design and requires only a simple DC voltage to activate. It is reset by a spring. In contrast, the DC gearmotor must have its polarity reversed to move the pin back to its original position. Control can be time- or current-based. [23, p. 3][The practical implementation is carried out using circuits with an H-bridge (in the case of the foccci board, it is a DRV8874 [25, p. 4]).

The secondhand i3 connector purchased includes an actuator from Küstner, consisting of a DC brushed motor, a gearbox, and a limit switch. In the absence of manufacturer-provided alternatives, an external circuit based on the (slightly oversized) L298N chip is used. While the locking motor has a typical operating current of 0.25 A, with peaks of up to 1.5 A, the L298N can operate continuously at 2 A. [23, p. 3] [26, p. 3]

Control is via 3 digital signals: IN1, IN2, ENA

Table 3 .1 : Bidirectional motor control, according to [26, p. 9]

Inputs		Function
ENA = H	IN1 = H; IN2 = L	Forward
	IN1 = L; IN2 = H	Reverse
	IN1 = IN2	Motor brake
ENA = L	IN1 = X; IN2 = X	Free-wheel

For use with the CCS connector, the ENA can be permanently connected to the supply voltage.³ A microswitch, which the Micro Controlling Unit (MCU) can read, serves as feedback. [23]

The L298N board used also includes an LDO that can be used to supply power to the rest of the board.

3.3.2 CP Measurement and Control

As mentioned in the chapter 2.1.2 , all communication in CCS takes place via the CP contact. Initially via a PWM signal; in the second step, a high-frequency signal is modulated over it. Accordingly, the vehicle-side controller has two tasks:

1. Measuring the PWM signal (duty cycle)⁴
2. Modifying the CP voltage level

Several protective measures are necessary when the CP signal is connected directly to a digital input of the processor. Firstly, the voltage levels are incompatible (5 V logic voltage at the MCU versus 12 V at CP)⁵ ; second, the total impedance of the vehicle must not be altered. Figure 3.3 in Figure 3 .3 at **Fehler! Verweisquelle konnte nicht gefunden werden.** Figure 3 .2 also comes from the admin panel. [49], [50]

3.3.3 MQTT Explorer

All EVerest communication runs through an MQTT broker (mosquitto). MQTT Explorer is an open-source tool for graphically visualizing MQTT communication. After connecting to the broker, all messages are sorted by topic and can be tracked over time.

Active intervention is also possible via MQTT Explorer. You can manually specify a topic and send a message with any attachment. Specifically: If you want to manually set the hardware's

³Motor speed is controlled via PWM on this pin. However, that is not necessary here.

⁴Voltage measurement is not required, as there is no hardware installed on the EVSE side to manipulate the CP voltage level.

⁵The negative component is blocked by the upstream diode.

state using the `ev\_board\_support` interface (e.g., `cp\_state: B`), you can publish the following message from MQTT Explorer:

```
topic:
everest/modules/DieterEvDriver_instance/impl/board_support/cmd/set_state_c

payload:
{
  "data": {
    "args": {
      "cp_state": "B"
    },
    "id": "372c0123-9b68-45e1-9255-8083ae4c0224",
    "origin": "manual test"
  },
  "msg_type": "Cmd"
}
```

This has the advantage of allowing for quick testing without having to implement every detail first.

3.3.4 Wireshark

Wireshark is free software for capturing and graphically analyzing data packets and has become the standard for network analysis. It supports a wide variety of interfaces, including the local network, Bluetooth, and USB.

With a suitable plugin, the entire HLC between the vehicle and the charging station can be decoded. Broken down by color, the individual phases can be easily identified. [51], [52]

One challenge in the PEV Simulator project, where Wireshark proved particularly useful, was troubleshooting the initial connection between the PLC modem and Linux. As soon as a physical connection is established, the Linux host attempts to assign an IP address to the Ethernet adapter via DHCP. Normally, this is the desired behavior and results in a quick connection setup. In this specific case, however, this is not the case, as the interface is actively managed by the software. The intervention of NetworkManager results in very long wait times and, in most cases, a connection failure. The following can then be observed on Linux:

```
gregor-sellschopp@GS:~/everest/NMA_PEV_SIM$ nmcli d
DEVICE          TYPE      STATE              CONNECTION
wlp4s0          wifi      verbunden         NMA_EVSE
enx000acd2d04f2 ethernet  wird verbunden (IP-Einstellungen werden ermittelt) Kabelgebundene Verbindung 2
br-0255813afcc4 bridge    verbunden (extern) br-0255813afcc4
br-c609e3492545 bridge    verbunden (extern) br-c609e3492545
lo              loopback  verbunden (extern) lo
docker0         bridge    verbunden (extern) docker0
p2p-dev-wlp4s0  wifi-p2p  nicht verbunden   --
enp0s31f6       ethernet  nicht verfügbar   --
cdc-wdm0        gsm       nicht verfügbar   --
tailscale0      tun       nicht verwaltet    --
```

Figure 3.3: Incorrect IP assignment by the Homeplug modem

This finding was revealed by Wireshark captures. The simple solution was to have NetworkManager ignore the interface using a configuration file.

```
# /etc/NetworkManager/conf.d/homeplug-unmanaged.conf
[keyfile]
unmanaged-devices=interface-name:<iface>
```

The MAC address can be determined using ``ip link show``. After that, all you need to do is restart NetworkManager and enable the interface.⁶

```
sudo systemctl restart NetworkManager
ip link set <iface> up
```

3.3.5 PlatformIO

There are many ways to program a microcontroller; the Arduino IDE is probably the simplest of them. PlatformIO works similarly but is significantly more flexible. At its core, it is a suite of tools for developing on embedded systems, enabling relatively hardware-independent development. Configuration is done (unlike with the Arduino IDE) via a .ini or YAML file.

PlatformIO is also used to program the Arduino in the “Dieter” electronics project. As a plugin for VS Code, it’s very easy to integrate into your workflow. [53], [54]

⁶ This final step is required after every connection restart. A corresponding cron script is recommended.

4 Results

When commissioning the hardware, all of its individual functions are tested first. When testing on a simple AC charging station, the PLC is not required, and the functionality of the CP MOSFET and the PWM measurement via `pulseIn` (see the section at **Fehler! Verweisquelle konnte nicht gefunden werden.**) can be verified. Since a DC charging station always operates at 5% PWM, an AC station is suitable because it allows the charging power to be varied and the measured values to be observed.

Voltage measurement via an external ADC and isolation chip does not work out of the box. Due to missing traces on the board, the internal DC-DC converter of the ISOW7441 isolation chip has an incorrect voltage reference, causing it to generate 8 V instead of 5 V and overheat. This high voltage also destroyed the ADC. The circuit diagrams will be corrected, but a second iteration will not be manufactured.

EVERest does not require the measured voltage values, so it is still possible to run a charging simulation.

The next step is DC charging. Here, negotiating the IP addresses (SLAC matching) is an essential part of establishing the connection before voltage can be measured at the Combo-2 connector. After the problem with the NetworkManager was resolved and the Homeplug modem was patched (see chapters 3.3.4 and **Fehler! Verweisquelle konnte nicht gefunden werden.**), this became reproducibly possible. Parameters for the load simulation can be successfully set in the runtime configuration (see the chapters at **Fehler! Verweisquelle konnte nicht gefunden werden.**).

The entire setup is mounted in a control cabinet, including a CCS connector. It is secured using a DIN rail. 3D-printed mounts are used for the SBC, Dieter, Homeplug modem, and motor driver.⁷

⁷ https://github.com/gregsell/Dieter_PEV_Simulator/tree/main/docs

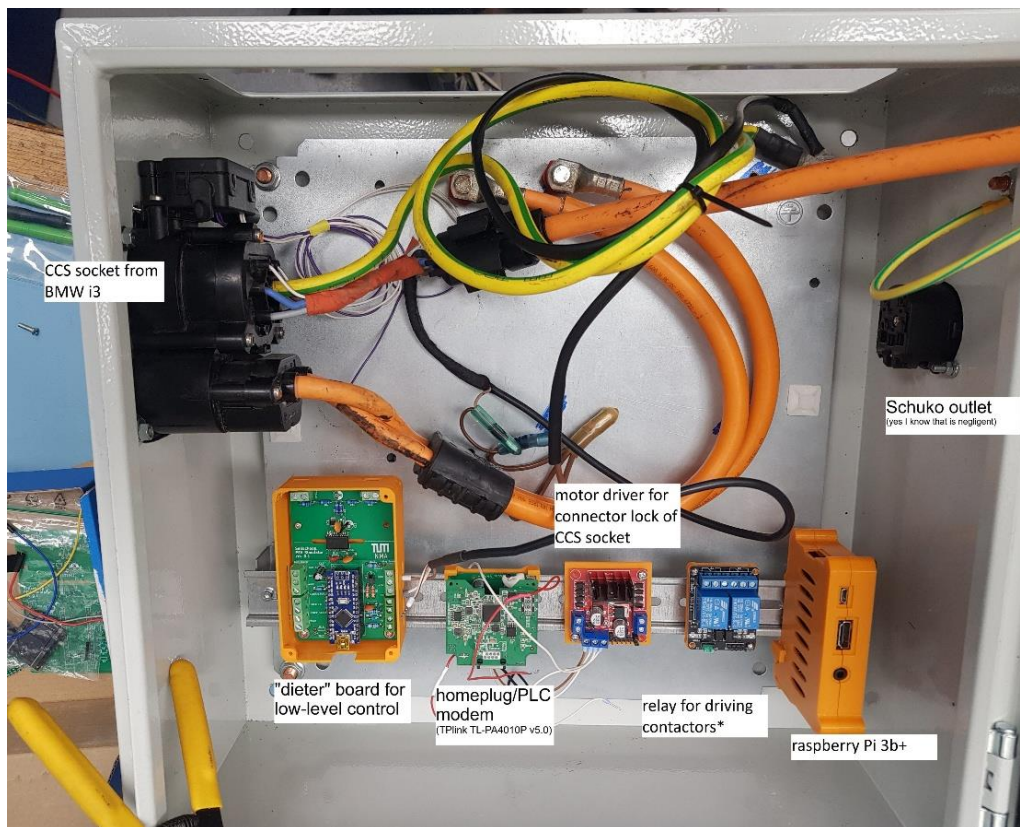


Figure 4 .1 Construction of the Vehicle Simulator

The DieterEvDriver module developed in this thesis, which provides the `ev_board_support` interface, performs its task without any issues during operation.

In certain edge cases, additional logic may be necessary. If the charging simulation is terminated unexpectedly because the EVERest process crashes, the behavior of the contactors and interlocks is not yet consistent. This is relevant not only in the event of a crash but also when the process is manually terminated in the terminal.

The source code for the “Dieter” firmware and the EVERest driver module are available on GitHub in their own respective repositories.⁸ Logs from DC charging simulations on both EVERest-based charging stations and public Alpitronic devices can be found on GitHub at [gregsell/DieterEvDriver/logs](https://github.com/gregsell/DieterEvDriver/logs). There you will find both successful and failed simulations, providing a reference for various error scenarios.

⁸ https://github.com/gregsell/Dieter_PEV_Simulator/, <https://github.com/gregsell/DieterEvDriver>



Figure 4.2 : Making tea at the Alpitronic HYC-300

To enable the EVerest project to run on the available Raspberry Pi, a new BitBake layer is written and a custom Linux distribution is created. The finished image is fully functional and can run EVerest projects on hardware with as little as 1 GB of RAM. Any additional referenced packages (including Java, Mosquitto, and tshark) work in conjunction with EVerest.

The BitBake configuration and image can be found in the DieterEvDriver repository under `/docs/yocto`. The fully compiled image has also been uploaded and is available under “releases.”

We had to work with Yocto because no suitable hardware was available. What started as a side project—intended merely as a “tool”—quickly became quite complex. Such a fully integrated Yocto workflow is primarily intended for production and isn’t worthwhile for a one-time setup; however, it is the right approach for a well-documented research setup.

5 Summary, Conclusions, and Outlook

This paper first explains the fundamentals of the CCS protocol and the connectors used. A distinction is made between analog communication via the PWM signal on the CP pin and the PLC, which is required for DC charging. Two existing CCS implementations are analyzed and compared in terms of flexibility.

Subsequently, requirements for the PEV simulator are defined, and the system architecture is outlined. The setup can be divided into two subprojects: first, the Linux system on which EVERest runs; and second, the microcontroller, including its firmware, which physically connects the setup to the CCS.

The schematic for the “Dieter” interface electronics is designed in KiCAD, while Platformio is used for firmware development.

In the context of EVERest, this thesis develops the “DieterEvDriver” hardware support module as well as functional runtime configurations for AC and DC charging. In a second step, a Linux image for the Raspberry Pi 3b+ is successfully generated using the Yocto Project, and the EVERest vehicle simulation is then run on this platform.

EVERest, as an ecosystem, is well-suited for developing a vehicle simulator and is fundamentally very flexible. As is often the case, however, this flexibility comes with complexity. Getting started is particularly complex due to the limited support and documentation that is not always consistent. Working with Yocto is instructive and time-consuming, but it is the right approach.

In some areas, the setup can still be improved in terms of safety and functionality. For example, the contactors remain closed if the EVERest process terminates. This can be resolved by a regular signal from the driver module to the MCU (“heartbeat”). As soon as the process crashes, the MCU can establish a safe operating state. Additionally, a power isolator along with high-power contactors can be installed to switch higher currents. The connector system at the output also does not meet the highest standards and can be improved.

Ultimately, this setup functioned very reliably, both on the EVERest-based charging station and in conjunction with Alpitronic devices.

6 References

- [1] Wikipedia. "Combined Charging System." Accessed April
- [2] evchargerpile. "Photo of Type 1 Combo CCS Plug." Accessed April 25, 2026. [Online.] Available at: <https://www.evchargerpile.com/de/video/products-detail-778739>
- [3] gcharin, "design_guide_combined_charging_system_v7," pp. 1–115, 2019. [Online.] Available at: https://www.charin.global/media/pages/technology/ccs-specification/c07034e41e-1626949173/design_guide_combined_charging_system_v7.pdf
- [4] chademo. "JAE | V2H KW02C | CHAdeMO." Accessed April 25, 2026. [Online.] Available at: <https://www.chademo.com/products/connectors/e-v2h>
- [5] Uwe Hennig, Johannes Hübner, Linux Info Tag Augsburg. "Open Source CCS Implementation for Using the Fast-Charging Infrastructure." Accessed April 25, 2026. [Online.] Available at: <https://www.youtube.com/watch?v=Bc7IXPGtLXY>
- [6] BNA. "Federal Network Agency – Charging Station Map." Accessed April 25, 2026. [Online.] Available at: <https://www.bundesnetzagentur.de/DE/Fachthemen/ElektrizitaetundGas/E-Mobilitaet/Ladesaeulenkarte/start.html>
- [7] Wikipedia. "GB/T charging standard." Accessed April 25, 2026. [Online.] Available at: https://en.wikipedia.org/w/index.php?title=GB/T_charging_standard&oldid=1310785854
- [8] C. e. V. "CharIN Stands Behind CCS and MCS, but Also Supports the Standardization of Tesla NACS – CharIN." Accessed April 25, 2026. [Online.] Available at: <https://www.charin.global/news/charin-stands-behind-ccs-and-mcs-but-also-supports-the-standardization-of-tesla-nacs/>
- [9] GitHub: Qualcomm Atheros. "qca/open-plc-utils: Qualcomm Atheros Open Powerline Toolkit." Accessed April 26, 2026. [Online.] Available at: <https://github.com/qca/open-plc-utils>
- [10] openinverter Forum. "Open source CCS using AR7420 - openinverter forum." Accessed April 26, 2026. [Online.] Available at: <https://openinverter.org/forum/viewtopic.php?p=37085#p37085>
- [11] GitHub. "uhi22/pyPLC: Hardware for CCS Sniffing." Accessed April 26, 2026. [Online.] Available at: <https://github.com/uhi22/pyPLC/blob/master/doc/hardware.md>
- [12] GitHub: uhi22. "pyPLC/doc at master · uhi22/pyPLC." Accessed April 24, 2026. [Online.] Available at: <https://github.com/uhi22/pyPLC/tree/master>

- [13] GitHub: uhi22. "uhi22/dieter: Device Interface ElecTronic—not limited to Raspberry." Accessed April
- [14] GitHub: uhi22. "uhi22/foccci: Fully Open CCS Charge Controller Interface with QCA7005 PLC Modem." Accessed April 26, 2026. [Online.] Available at: <https://github.com/uhi22/foccci>
- [15] openinverter shop. "openinverter shop: foccci." Accessed April
- [16] everest.
- [17] mintlify: EVerest Core. "Interface System - EVerest Core." Accessed April 26, 2026. [Online.] Available at: <https://mintlify.wiki/EVerest/everest-core/core-concepts/interfaces>
- [18] everest.
- [19] mintlify: everest modules, "Modules Overview," *EVerest Core*. Accessed April 27, 2026. [Online.] Available at: <https://mintlify.wiki/EVerest/everest-core/modules/overview>
- [20] everest.github.io. "PhyVersoBSP - EVerest Manual 2026.02.0." Accessed April 27, 2026. [Online.] Available at: <https://everest.github.io/nightly/reference/modules/HardwareDrivers/EVSE/PhyVersoBSP/autogenerated.html>
- [21] everest.
- [22] "TIDA-010939 Reference Design | TI.com." Accessed April 29, 2026. [Online.] Available at: <https://www.ti.com/tool/de-de/TIDA-010939>
- [23] "Phoenix Contact CCS Socket Datasheet." Accessed May 3, 2026. [Online.] Available: https://www.phoenixcontact.com/product/pdf/api/v1/MTcyMDA5OA?_realm=us&_locale=en-US&blocks=commercial-data%2Ctechnical-data%2Cdrawings%2Cclassifications%2Cenvironmental-compliance-data
- [24] DigiKey Electronics. "CCS socket | DigiKey Electronics." Accessed May 3, 2026. [Online.] Available at: <https://www.digikey.de/de/products/detail/phoenix-contact/1720098/25576714>
- [25] "foccci schematic," [Online]. Available at: https://github.com/uhi22/foccci/blob/main/pdf/foccci_schematic_v5.2p7.pdf
- [26] STMicroelectronics. "L298 Datasheet." Accessed May 3, 2026. [Online.] Available at: www.st.com/resource/en/datasheet/l298.pdf
- [27] Texas Instruments, Incorporated [SLLA353 and A], "Isolation Glossary (Rev. A)," [Online]. Available at: https://www.ti.com/lit/an/slla353a/slla353a.pdf?ts=1777703378173&ref_url=https%253A%252F%252Fti.com%252Flit%252Fpdf%252Fslla353

- [28] "ACS712 Datasheet," [Online]. Available at: <https://www.allegromicro.com/-/media/files/datasheets/acs712-datasheet.ashx>
- [29] Texas Instruments, Incorporated [SBOSAF5 and B], "TMCS1126-Q1 AEC-Q100, Precision 500 kHz Hall-Effect Current Sensor With Reinforced Working Voltage, Overcurrent Detection, and Ambient Field Rejection datasheet (Rev. B)," [Online]. Available at: <https://www.ti.com/lit/ds/symlink/tmcs1126-q1.pdf?ts=1777645502209>
- [30] Texas Instruments, Incorporated [SBAS740 and B], "ADS1118-Q1 Automotive, Low-Power, SPI-Compatible, 16-Bit Analog-to-Digital Converter with Internal Reference and Temperature Sensor datasheet (Rev. B)," [Online]. Available at: <https://www.ti.com/lit/ds/symlink/ads1118-q1.pdf?ts=1777717695847>
- [31] Texas Instruments, Incorporated [SLLSFK3 and *], "ISOW774x-Q1 Quad-Channel Digital Isolator with Integrated Low-Emissions, Low-Noise DC-DC Converter datasheet," [Online]. Available at: <https://www.ti.com/lit/ds/symlink/isow7741-q1.pdf?ts=1777707377926>
- [32] "Protron DC Circuit Breakers, MCBs, 4-pole and 2-pole, Circuit Breakers for Solar PV Systems 2P, 4P, 600V, 1000V, 10–63A, Polarity-Independent | plentyShop LTS." Accessed May 4, 2026. [Online.] Available at: https://www.arcs-europe.de/home-automation/solar-technology/accessories/protron-dc-circuit-breakers-ls-switches-mcb-circuit-breakers-for-solar-pv-photovoltaic-systems-2p-4p_1291_2772
- [33] "Installation Guide," *EVERest Core*. Accessed May 4, 2026. [Online.] Available at: <https://mintlify.wiki/EVERest/everest-core/installation#prerequisites>
- [34] Electronic Systems Design. "PCB High-Voltage Spacing: What Every Engineer Should Know - Electronic Systems Design." Accessed May 7, 2026. [Online.] Available at: <https://blogs.sw.siemens.com/electronic-systems-design/2025/04/29/pcb-high-voltage-spacing-what-every-engineer-should-know/>
- [35] Wikipedia. "Consistent Overhead Byte Stuffing." Accessed May 10, 2026. [Online.] Available at: https://en.wikipedia.org/w/index.php?title=Consistent_Overhead_Byte_Stuffing&oldid=1344930661
- [36] GitHub. "EVERest: YetiEvDriver." Accessed May 10, 2026. [Online.] Available at: <https://github.com/EVERest/EVERest/tree/main/modules/HardwareDrivers/EV/YetiEvDriver>
- [37] GitHub. "EVERest - EvManager source." Accessed May 11, 2026. [Online.] Available at: <https://github.com/EVERest/EVERest/tree/main/modules/EV/EvManager>
- [38] "ev_board_support - EVERest Manual 2026.02.0." Accessed May 11, 2026. [Online.] Available at: https://everest.github.io/nightly/reference/interfaces/ev_board_support.html
- [39] P. P. "Raspberry Pi cross development." Accessed May 18, 2026. [Online.] Available at: <https://tttapa.github.io/Pages/Raspberry-Pi/>

- [40] GitHub. "ttroy50/cmake-examples: Useful CMake Examples." Accessed May 18, 2026. [Online.] Available at: <https://github.com/ttroy50/cmake-examples>
- [41] GitHub. "tttapa/docker-arm-cross-toolchain: Modern GCC cross-compilation toolchains for Raspberry Pi." Accessed May 18, 2026. [Online.] Available at: <https://github.com/tttapa/docker-arm-cross-toolchain>
- [42] Wikipedia. "CMake." Accessed May 18, 2026. [Online.] Available at: <https://de.wikipedia.org/w/index.php?title=CMake&oldid=262443251>
- [43] YouTube. "How to CMake Good - Recommended Order." Accessed May 18, 2026. [Online.] Available at: <https://www.youtube.com/playlist?list=PLK6MXr8gasrGmliSu-VQXpfFuE1uPT615s>
- [44] "Development — Charge Control C User Guide." Accessed May 31, 2026. [Online.] Available at: https://chargebyte-docs.readthedocs.io/projects/everest-charge-control-c/en/everest_charge_control_c_1.6.0/development.html
- [45] YouTube. "Grazer Linuxtage 2021 - Yocto Crash Course." Accessed June 1, 2026. [Online.] Available at: <https://www.youtube.com/watch?v=8z9DpS5rUA>
- [46] "Linux / Yocto - EVerest Manual 2026.02.0." Accessed May 31, 2026. [Online.] Available at: <https://everest.github.io/nightly/explanation/linux-yocto/index.html>
- [47] "Yocto Linux Integration," *EVerest Core*. Accessed May 31, 2026. [Online.] Available at: <https://mintlify.wiki/EVerest/everest-core/deployment/yocto>
- [48] "ev-cli - EVerest Manual 2026.02.0." Accessed May 18, 2026. [Online.] Available at: <https://everest.github.io/nightly/explanation/dev-tools/ev-cli.html#ev-cli>
- [49] GitHub. "EVerest/everest-admin-panel." Accessed April 29, 2026. [Online.] Available at: <https://github.com/EVerest/everest-admin-panel>
- [50] "Get Started in Software - EVerest Manual 2026.02.0." Accessed May 18, 2026. [Online.] Available at: <https://everest.github.io/nightly/how-to-guides/getting-started/get-started-sw.html>
- [51] GitHub. "dspace-group/dsV2Gshark: Free Wireshark plugin to analyze ISO 15118 V2G communication between electric vehicles (EV) and charging stations (EVSE)." Accessed May 19, 2026. [Online.] Available at: <https://github.com/dspace-group/dsV2Gshark>
- [52] Wikipedia. "Wireshark." Accessed May 19, 2026. [Online.] Available at: <https://de.wikipedia.org/w/index.php?title=Wireshark&oldid=257457503>
- [53] PlatformIO. "PlatformIO: Your Gateway to Embedded Software Development Excellence." Accessed May 19, 2026. [Online.] Available at: <https://platformio.org/>

- [54] Wikipedia. "PlatformIO." Accessed May 19, 2026. [Online.] Available at: <https://de.wikipedia.org/w/index.php?title=PlatformIO&oldid=256749101>
- [55] GitHub. "dspace-group/dsV2Gshark: Free Wireshark plugin to analyze ISO 15118 V2G communication between electric vehicles (EV) and charging stations (EVSE)." Accessed on: June 1, 2026. [Online.] Available at: https://github.com/dspace-group/dsV2Gshark/tree/main/Wireshark/dsV2Gshark_examples

Appendix

Appendix A	Runtime Configuration.....	50
Appendix B	Dieter's Circuit Diagram.....	52
Appendix C	Dieter Circuit Board	53
Appendix D	Yocto	54

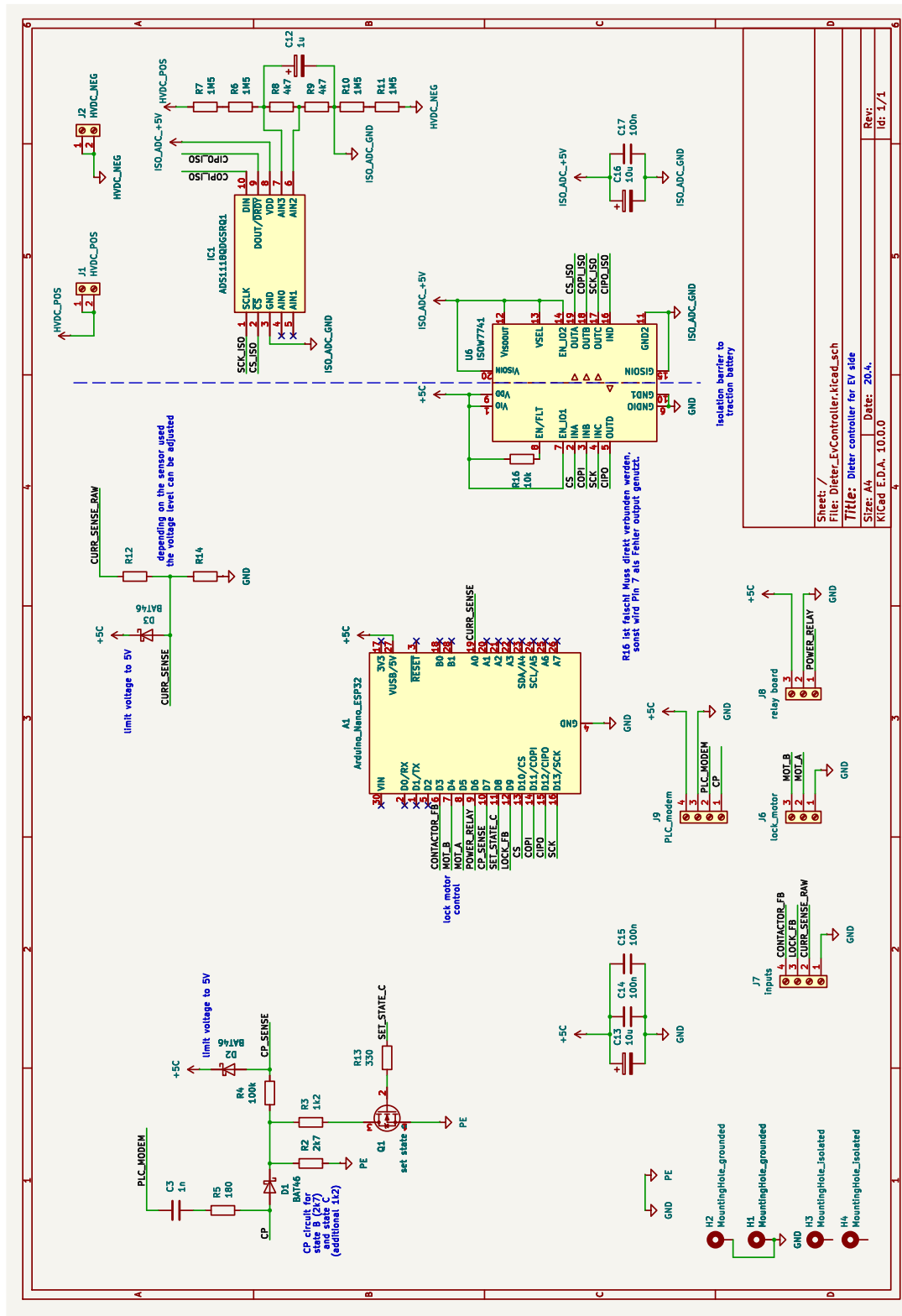
Appendix A Runtime Configuration

Runtime configuration for a DC charging simulation. The sequence is defined using `auto_exec_commands`

```
settings:
  telemetry_enabled: true
active_modules:
  DieterEvDriver_instance:
    module: DieterEvDriver
    connections: {}
    config_module:
      baud_rate: 19200
      serial_port: /dev/ttyUSB0
  EvManager0:
    module: EvManager
    connections:
      ev:
        - module_id: PyEvJosev0
          implementation_id: ev
      ev_board_support:
        - module_id: DieterEvDriver_instance
          implementation_id: board_support
    kvs:
      - module_id: Store0
        implementation_id: main
    slac:
      - module_id: EvSlac0
        implementation_id: main
    config_module:
      ac_nominal_voltage: 230
      auto_enable: true
      auto_exec: true
      auto_exec_commands: 'sleep 2;iso_wait_pwm_is_run-
ning;iso_wait_slac_matched;iso_start_v2g_session
DC;iso_wait_pwr_ready;iso_dc_power_on;iso_wait_for_stop 120;unplug'
      auto_exec_infinite: false
      connector_id: 1
      dc_discharge_max_current_limit: 300
      dc_discharge_max_power_limit: 150000
      dc_discharge_target_current: 5
      dc_discharge_v2g_minimal_soc: 20
      dc_energy_capacity: 60,000
      dc_max_current_limit: 300
      dc_max_power_limit: 150,000
      dc_max_voltage_limit: 900
      dc_target_current: 5
      dc_target_voltage: 200
      departure_time: 86400
      e_amount: 0
      force_payment_option: false
      keep_cross_boot_plugin_state: false
      max_current: 16
      plugin_commands: ''
      soc: 30
      support_sae_j2847: false
      three_phases: false
  PyEvJosev0:
    module: PyEvJosev
    connections: {}
    config_module:
      device: eth0
      enable_tls_1_3: false
      enforce_tls: false
      is_cert_install_needed: false
      is_internet_service_needed: false
      request_all_service_details: false
```

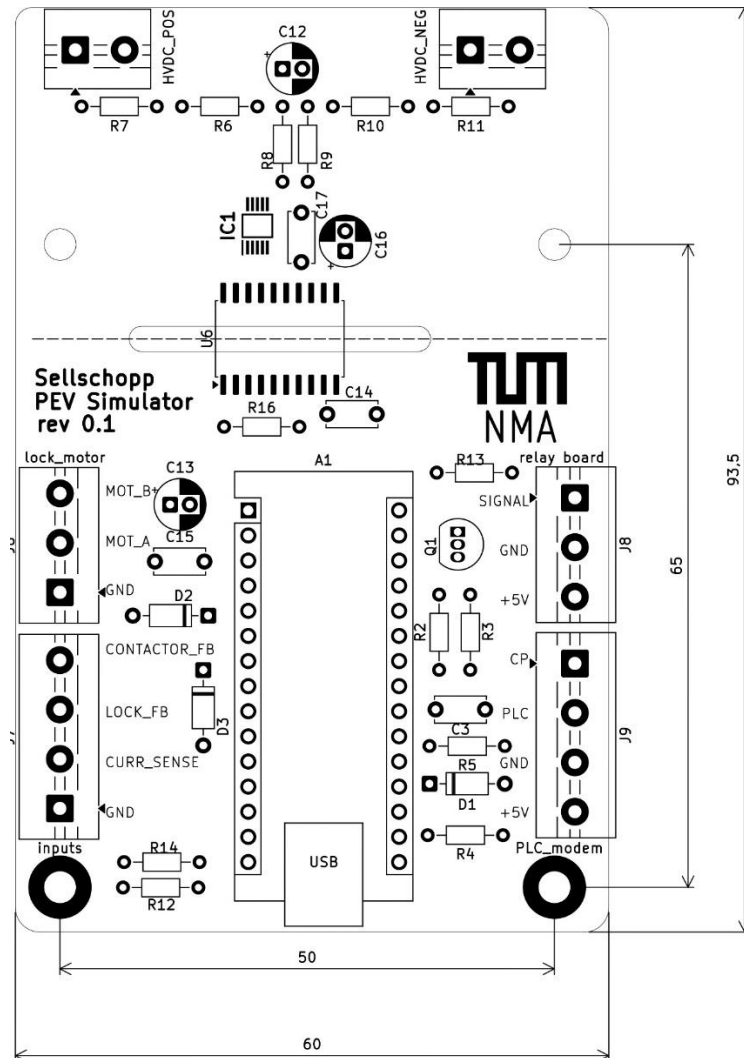
```
select_all_vas_services: false
supported_DIN70121: false
supported_ISO15118_2: true
supported_ISO15118_20_AC: false
supported_ISO15118_20_DC: true
supported_d20_energy_services: DC,DC_BPT
tls_active: false
Store0:
  module: Store
  connections: {}
EvSlac0:
  module: EvSlac
  connections: {}
  config_implementation:
    main:
      device: eth0
      set_key_timeout_ms: 500
```

Appendix B Dieter's Schematic



FigureB .1 : Dieter's Schematic

Appendix C Dieter PCB



FigureC .1 PCB Dimensions

Appendix D Yocto

conf/local.conf:

```
1. MACHINE = "raspberrypi3-64"
2.
3. # Parallelization (adjust to your CPU)
4. BB_NUMBER_THREADS = "10"
5. PARALLEL_MAKE = "-j 10"
6.
7. # Disk space monitoring
8. BB_DISKMON_DIRS = "STOPTASKS,${TMPDIR},1G,100K STOPTASKS,${DL_DIR},1G,100K"
9.
10. # Add Everest to the image
11. IMAGE_INSTALL:append = "everest-core mosquitto python3 kernel-module-cdc-acm kernel-module-ch341 kernel-module-usbserial dieter-ev-driver python3-iso15118 openjdk-17-jre tshark"
12.
13.
14. ENABLE_UART = "1"
15.
16. DISTRO_FEATURES:append = "systemd usrmerge wifi"
17. INIT_MANAGER = "systemd"
18.
19. LICENSE_FLAGS_ACCEPTED = "synaptics-killswitch"
20.
21. EVEREST_CORE_PATH = "/path/to/everest"
22. BBMASK += "meta-everest/recipes-connectivity/openssl/"
23.
24. # Common download cache for all builds
25. DL_DIR = "/some/shared/directory"
26.
27. # Common sstate cache (reusable build artifacts)
28. SSTATE_DIR = "/another/shared/directory"
29.
30. # root login without a password
31. EXTRA_IMAGE_FEATURES += "debug-tweaks"
```